
Cómputo Nube y Tecnologías Emergentes

Instituto Nacional de Astrofísica, Óptica y Electrónica (INAOE) - Maestría en Ciencias en Tecnologías de Seguridad
Lázaro Bustio-Martínez, PhD
lbustio@inaoe.mx
Verano 2026
96 horas
- Guía de Laboratorio Práctico -

Guía de Laboratorio Operativo y Reproducible Detección de Fuerza Bruta mediante Análisis Estructurado de Logs y Elasticsearch REST API

1. Introducción: De la Observabilidad al SIEM

En la ingeniería cloud real, los sistemas nunca funcionan a la perfección en el primer intento. Levantar la infraestructura es apenas el 20 % del trabajo; el 80 % restante consiste en modelar los datos, resolver bloqueos de red y lidiar con excepciones de bases de datos.

Objetivo del laboratorio: Construir un pipeline que transite desde la recolección cruda de logs de Linux hasta la visualización analítica. A lo largo del ejercicio, se enfrentarán los mismos problemas que experimenta un ingeniero en un Centro de Operaciones de Seguridad (SOC): caídas por falta de memoria, bloqueos de firewall y colisiones de tipos de datos.

Arquitectura y Resultado: Utilizaremos Python para leer bitácoras de seguridad en GCP, las inyectaremos en Elasticsearch a través de su API REST, y utilizaremos Kibana para aislar patrones de ataques automatizados (fuerza bruta). Aunque esto establece las bases de la **Observabilidad**, sentará los cimientos para entender cómo opera un **SIEM** (Security Information and Event Management) real al correlacionar eventos estructurados.

2. Fase 1: Configuración de Red e Infraestructura Cloud

2.1. Paso 1.1: Apertura de Puertos Perimetrales (GCP Firewall)

- **Objetivo del paso:** Configurar las políticas de red externas (*Ingress Rules*) en Google Cloud Platform.
- **Por qué se hace:** Esto se hace para permitir que agentes externos despachen datos HTTP hacia la API de Elasticsearch y que los analistas accedan a la consola web de Kibana desde el exterior.
- **Qué problema resuelve:** Resuelve el aislamiento por defecto de la infraestructura de nube, que bloquea todo el tráfico entrante a las instancias de cómputo aplicando un modelo *Zero Trust*. Si no abrimos los puertos, las peticiones HTTP morirán por *Timeout*.
- **Acción concreta:** Abra la terminal de comandos de Google Cloud SDK (`gcloud CLI`) en su máquina de administración local o use la consola web de GCP navegando a *Red de VPC -> Firewall*. Ejecute las siguientes reglas de filtrado perimetral:

```
# 1. Crear regla de firewall perimetral para permitir acceso a la interfaz web de Kibana
gcloud compute firewall-rules create allow-kibana-ingress \
```

```
--allow=tcp:5601 \
--description="Acceso publico a la interfaz web de Kibana para analistas" \
--direction=INGRESS \
--priority=1000 \
--network=default \
--target-tags=elk-server
```

```
# 2. Crear regla de firewall perimetral para permitir consultas e ingesta HTTP REST
gcloud compute firewall-rules create allow-elasticsearch-rest \
```

```
--allow=tcp:9200 \
--description="Acceso externo a la API REST de Elasticsearch" \
--direction=INGRESS \
--priority=1000 \
--network=default \
--target-tags=elk-server
```

Posteriormente, asigne manualmente la etiqueta de red `elk-server` a su Instancia de Máquina Virtual de Compute Engine mediante la consola de GCP o ejecutando:

```
gcloud compute instances add-tags <NOMBRE_DE_TU_VM> --tags=elk-server --zone=<TU_ZONA_DE_
```

- **Explicación técnica inmediata:** Estos comandos inyectan reglas de filtrado en la red definida por software (SDN) de GCP. Al especificar `-allow=tcp:5601` y `tcp:9200`, el enrutador de GCP inspecciona el encabezado de transporte de las peticiones entrantes y levanta el bloqueo físico. Al usar `-target-tags=elk-server`, la regla no se aplica globalmente a toda la nube, sino únicamente a las VMs que contengan dicha propiedad semántica.
- **Resultado esperado:** La terminal del SDK debe retornar una tabla con el estado de creación de la regla. Se valida que funcionó si la columna `STATUS` muestra `READY` y los puertos aparecen asignados correctamente. Significa que el perímetro ya acepta conexiones.
- **Troubleshooting:** Si recibe el error `ERROR: (gcloud.compute.firewall-rules.create) Resource already exists`, significa que la regla ya fue declarada previamente; soluciónelo cambiando el nombre o ejecutando un comando `delete` primero. Si al intentar conectarse experimenta un retraso prolongado seguido de `ERR_CONNECTION_TIMED_OUT`, la regla no se ha propagado o se equivocó al escribir la etiqueta de red `elk-server` en la VM; verifique la correspondencia exacta de caracteres usando `gcloud compute instances describe <NOMBRE_DE_TU_VM>`.

3. Fase 2: Despliegue de la Infraestructura ELK (Docker Compose)

3.1. Paso 2.1: Clonación e Inicialización del Stack Aislado

- **Objetivo del paso:** Descargar el entorno preconfigurado de contenedores Docker e inicializar Elasticsearch y Kibana.
- **Por qué se hace:** Esto se realiza para instanciar microservicios repetibles que no dependan de la configuración interna del sistema operativo base.
- **Qué problema resuelve:** Resuelve el "infierno de dependencias" (*dependency hell*) y garantiza consistencia de versiones entre entornos de desarrollo y producción.
- **Acción concreta:** Conéctese vía SSH a su máquina virtual de GCP. Instálese en su directorio de usuario (`~/`) y ejecute exactamente la siguiente secuencia de comandos en su terminal Linux:

```
sudo apt-get update && sudo apt-get install -y git docker.io docker-compose-v2
git clone https://github.com/deviantony/docker-elk.git
cd docker-elk
docker compose up -d
```

- **Explicación técnica inmediata:** `docker compose up -d` lee el archivo de manifiesto `docker-compose.yml`, descarga las imágenes oficiales de Elastic desde su registro central, asigna las variables de entorno, crea una red virtual aislada dentro del motor de Docker y arranca los procesos como demonios (`-d`) en segundo plano.
- **Resultado esperado:** En la terminal se observará la descarga de las capas del contenedor seguida de la confirmación: `Container docker-elk-elasticsearch-1 Started` y `Container docker-elk-kibana-1 Started`. Esto valida el arranque inicial de los procesos.
- **Troubleshooting:** Si aparece el error `docker: command not found`, es porque su VM no tiene instalado el motor de contenedores; asegúrese de haber corrido correctamente la línea de instalación con `apt-get`. Si la inicialización arroja un fallo de memoria y los procesos se mueren repentinamente, es porque Elasticsearch requiere mínimo 2GB de RAM libres. El Kernel de Linux mata el proceso lanzando una excepción (*Out Of Memory - Exit 137*). Solúcelo incrementando la capacidad de su VM en GCP a un tipo de instancia más robusto (ej. `e2-medium`).

3.2. Paso 2.2: Auditoría Operativa de Sockets de Red de los Contenedores

- **Objetivo del paso:** Auditar las interfaces de red del sistema operativo anfitrión para validar que los puertos internos de los contenedores están expuestos al tráfico externo.
- **Por qué se hace:** Esto se hace para certificar que el proceso no está atrapado en la interfaz de bucle cerrado local (*localhost*).
- **Qué problema resuelve:** Resuelve problemas donde la aplicación funciona internamente dentro del servidor pero es inaccesible desde el exterior para el agente de Python o el analista.
- **Acción concreta:** Ejecute los comandos de inspección de sockets directamente en la terminal de la VM:

```
# Listar los contenedores en ejecucion y constatar el mapeo de puertos de red
docker ps
```

```
# Inspeccionar que sockets TCP estan en estado LISTEN en el Kernel de Linux
ss -tulpn | grep -E "9200|5601"
```

- **Explicación técnica inmediata:** El comando `ss -tulpn` interroga directamente al subsistema de red del Kernel de Linux. La bandera `-t` filtra sockets TCP, `-u` UDP, `-l` sockets escuchando, `-p` muestra el proceso y `-n` fuerza la salida numérica en lugar de resolver nombres de servicios.

- **Resultado esperado:** La salida de `ss` debe mostrar cadenas del tipo `0.0.0.0:9200` o `*:5601`. Esto significa que los procesos están vinculados a la interfaz comodín (`*any interface*`), listos para recibir paquetes provenientes de cualquier dirección IP pública a través del router de GCP (interfaz física `eth0`).
- **Troubleshooting:** Si la columna de dirección local muestra `127.0.0.1:9200`, significa que Elasticsearch está amarrado localmente (`*loopback*`) y rechazará cualquier conexión externa por diseño de seguridad. Para resolver esto, detenga los contenedores con `docker compose down`, abra el archivo de configuración `elasticsearch.yml` ubicado dentro del directorio `config/` del proyecto clonado, y asegúrese de que la directiva `network.host` esté configurada como `0.0.0.0`. Luego vuelva a ejecutar el contenedor.

3.3. Paso 2.3: Validación de Salud del Motor mediante Consultas REST HTTP

- **Objetivo del paso:** Verificar el estado lógico interno de Elasticsearch interactuando con su API REST.
- **Por qué se hace:** Esto se hace para certificar que el motor de base de datos está operativo y listo para procesar transacciones JSON.
- **Qué problema resuelve:** Resuelve la incertidumbre de asumir que un proceso está sano simplemente porque su contenedor de Docker dice `Up`.^{en} la terminal.
- **Acción concreta:** Desde su computadora local (no dentro de la VM), abra una terminal y envíe una petición HTTP dirigida al nodo mediante `curl`, reemplazando el marcador por la IP pública asignada a su instancia de Google Cloud:

```
curl -X GET "http://<IP_PUBLICA_TU_VM_GCP>:9200/_cluster/health?pretty"
```

- **Explicación técnica inmediata:** Elasticsearch expone todas sus funcionalidades administrativas a través de una API RESTful sobre HTTP. Al enviar un verbo `GET` al endpoint `/_cluster/health`, se acciona un módulo interno que evalúa el estado de consistencia del almacenamiento del clúster y devuelve un documento estructurado.
- **Resultado esperado:** Recibirá un payload JSON formateado de manera legible debido al parámetro `pretty`. Debe enfocarse en la clave `"status"`, la cual mostrará el valor `yellow`.

Interpretación Pedagógica del Estado "Yellow"

¿Qué significa exactamente este resultado? En Elasticsearch, un estado **yellow** NO implica un fallo operativo catastrófico. Significa que **todos los fragmentos primarios (*shards*) han sido asignados y son funcionales**, por lo que el clúster puede indexar y responder consultas. Sin embargo, debido a que este laboratorio opera en una arquitectura de nodo único, Elasticsearch carece de un segundo nodo físico donde replicar copias de seguridad de los fragmentos. Al no poder ubicar los fragmentos de réplica, el motor emite una alerta preventiva cambiando el color de verde a amarillo. Es el estado esperado para entornos académicos y de desarrollo.

- **Troubleshooting:** Si el comando `curl` se queda colgado indefinidamente, ejecute localmente un comando de diagnóstico de red de bajo nivel: `nc -zv <IP_PUBLICA_VM>9200`. Si este último arroja un error de pérdida de conexión, el problema reside en el firewall de GCP (Fase 1); si el puerto responde pero `curl` devuelve una respuesta vacía o un código de error de autenticación, la regla de firewall de la Fase 1 falló o cambió los parámetros de seguridad predeterminados del stack.

Checkpoint de Validación 1

- ✓ Reglas de firewall aplicadas correctamente en la consola de GCP.
- ✓ Contenedores de Docker en ejecución verosímil (`docker ps`).
- ✓ Elasticsearch responde peticiones HTTP externas con estado **yellow**.

4. Fase 3: Modelado Semántico de Datos (Mappings Explícitos vía API REST)

4.1. Paso 3.1: Creación Estructurada del Índice con Tipos de Datos Tipificados

- **Objetivo del paso:** Declarar formalmente el esquema de almacenamiento de datos (*Mapping*) en Elasticsearch antes de la ingesta.
- **Por qué se hace:** Esto se realiza para predefinir de manera estricta cómo debe indexarse semánticamente cada campo de un registro de logs Linux.
- **Qué problema resuelve:** Resuelve el riesgo del mapeo dinámico (*Dynamic Mapping*), donde Elasticsearch adivina el tipo de dato basándose en el primer documento recibido. Si el primer log registra un texto plano que parece número y un log subsiguiente introduce caracteres alfabéticos, el motor arroja una excepción crítica `document_parsing_exception`, bloquea la ingesta y descarta el evento de seguridad de forma definitiva.
- **Acción concreta:** Abra la terminal de comandos o use la consola de herramientas de desarrollo de Kibana (*Dev Tools*). Envíe una petición HTTP con el método estructurado **PUT** para inicializar el índice `gcp-vm-logs` con el esquema exacto detallado abajo:

```
PUT /gcp-vm-logs
{
  "mappings": {
    "properties": {
      "@timestamp": { "type": "date" },
      "log_file": { "type": "keyword" },
      "hostname": { "type": "keyword" },
      "process": { "type": "keyword" },
      "event_type": { "type": "keyword" },
      "username": { "type": "keyword" },
      "source_ip": { "type": "ip" },
      "message": { "type": "text" }
    }
  }
}
```

■ **Explicación técnica inmediata:**

- **¿Por qué existe el tipo `text`?** El campo `message` guarda la línea completa y cruda del log. Al tipificarlo como `text`, Elasticsearch pasa la cadena por un analizador sintáctico lingüístico que rompe el párrafo en componentes individuales (`*tokens*`) y los guarda en un índice invertido. Esto permite realizar búsquedas complejas por aproximación de términos. El tipo `text` consume alta memoria y no permite realizar agregaciones matemáticas ni agrupaciones de gráficos.
- **¿Por qué existe el tipo `keyword`?** Los campos como `username`, `event_type` y `process` se configuran como `keyword`. Este tipo de dato le ordena a Elasticsearch saltarse el análisis lingüístico y almacenar el valor crudo como una cadena exacta. Es el tipo de dato obligatorio e indispensable para realizar agregaciones, agrupaciones métricas e histogramas analíticos en Kibana.
- **¿Por qué existe el tipo `ip`?** El campo `source_ip` almacena la dirección de red del atacante. Tipificarlo formalmente como `ip` permite que Elasticsearch valide estructuralmente direcciones IPv4 e IPv6, habilitando la capacidad avanzada de realizar búsquedas por rangos de subred (notación CIDR, ej: `45.148.10.0/24`). Si se guardara como texto simple, sería imposible filtrar rangos geográficos de red de manera eficiente.

- **Resultado esperado:** Elasticsearch debe responder con un mensaje JSON de confirmación: `.acknowledged: true`. Esto valida que el espacio de almacenamiento lógico ha sido reservado y estructurado de forma inmutable bajo las restricciones de tipos definidas.
- **Troubleshooting:** Si recibe el error `resource_already_exists_exception`, significa que el índice ya fue creado (posiblemente por mapeo dinámico erróneo al experimentar previamente). Para solu-

cionarlo y aplicar este mapping estricto, limpie la base de datos eliminando el índice viejo ejecutando primero `DELETE /gcp-vm-logs` y reenvíe la petición PUT.

Checkpoint de Validación 2

- ✓Índice `gcp-vm-logs` creado exitosamente de forma explícita.
- ✓Esquema de datos (Mappings) inmutable aplicado sin errores en la API REST.

5. Fase 4: Desarrollo Modular del Agente de Ingestión en Python

Para evitar la complejidad innecesaria de configurar transformaciones en Logstash, programaremos nuestro propio agente liviano en Python. A continuación se desglosa el script bloque por bloque funcional con fines puramente pedagógicos. El estudiante deberá unir estos bloques dentro de un único archivo llamado `log_shipper.py` en su máquina virtual de GCP.

5.1. Paso 4.1: Declaración de Dependencias e Inicialización del Contexto Operativo

- **Objetivo del paso:** Importar los módulos del sistema nativos de Python y declarar las variables globales que definen los endpoints de red.
- **Acción concreta:** Abra un editor de código en la VM (ej: `nano log_shipper.py`) e inicie el archivo con las siguientes líneas de código:

```
import os
import re
import requests
import json
from datetime import datetime

# Definicion de constantes globales del entorno operativo
ELASTIC_URL = "http://localhost:9200/gcp-vm-logs/_doc"
LOG_DIR = "/var/log"
HEADERS = {"Content-Type": "application/json"}
```

- **Explicación técnica inmediata:** El módulo `os` permite iterar sobre las rutas del sistema de archivos de Linux. `re` habilita el motor de expresiones regulares. `requests` gestiona los sockets cliente para enviar paquetes HTTP. La constante `ELASTIC_URL` apunta de manera explícita al endpoint de inserción de documentos unitarios (`/_doc`).

5.2. Paso 4.2: Implementación de Filtros de Redundancia y Protección de Datos Binarios

- **Objetivo del paso:** Crear una función de inspección previa antes de abrir cualquier archivo en disco.
- **Qué problema resuelve:** Resuelve problemas críticos de caída de scripts cuando el recolector intenta leer archivos de logs binarios corruptos o rotados (como `/var/log/wtmp` o `btmpt`), los cuales no son texto ASCII plano y revientan la codificación.
- **Acción concreta:** Agregue de manera continua el siguiente bloque lógico al archivo en edición:

```
def is_binary(file_path):  
    """ Evalua la cabecera del archivo para descartar binarios que corrompen el flujo de  
    try:  
        with open(file_path, 'tr') as f:  
            f.readline() # Intentar leer la primera linea en formato de texto plano  
            return False  
    except UnicodeDecodeError:  
        # El Kernel lanza esta excepcion si encuentra bytes no mapeados en UTF-8 o ASCII  
        return True
```

- **Explicación técnica inmediata:** La función abre el archivo en modo de lectura de texto (`'tr'`). Si el archivo contiene datos binarios estructurados de bajo nivel del sistema, el decodificador de Python falla inmediatamente al intentar interpretar los bytes como caracteres legibles, lanzando una excepción `UnicodeDecodeError`, lo que nos permite saltarnos el archivo de forma segura.

5.3. Paso 4.3: Definición del Motor de Expresiones Regulares (Regex) para Parsing de Syslog

- **Objetivo del paso:** Compilar un patrón de expresiones regulares para deconstruir líneas de texto plano crudo en un diccionario de variables nombradas bien estructurado.
- **Acción concreta:** Inyecte la expresión regular de compilación global en su archivo:

```
# Compilar expresion regular nativa con grupos nombrados (?P<nombre>)  
LOG_PATTERN = re.compile(  
    r'(?P<month>\w{3})\s+'      # Grupo month: Tres letras del mes (ej: May)  
    r'(?P<day>\d+)\s+'         # Grupo day: Uno o mas digitos del dia (ej: 23)  
    r'(?P<time>\d+:\d+:\d+)\s+' # Grupo time: Formato de reloj de horas:minutos:segundo  
    r'(?P<hostname>[\w-]+\s+'  # Grupo hostname: Nombre alfabetico del servidor Linux  
    r'(?P<process>[\w-]+\s+)[\d+]:\s+' # Grupo process: Nombre del servicio omitiendo el P  
    r'(?P<message>.*)'        # Grupo message: Captura codificada de toda la cadena r  
)
```

- **Explicación técnica inmediata:** El motor Regex de Python analiza la cadena de izquierda a derecha. Al usar la bandera interna (?P<name>...), el motor mapea dinámicamente el fragmento extraído a una clave de diccionario accesible en memoria, estructurando texto plano sin procesar en variables semánticas explícitas.

5.4. Paso 4.4: Lógica de Clasificación de Ciberseguridad (SIEM) y Construcción del Objeto JSON

- **Objetivo del paso:** Analizar semánticamente el mensaje interno del log en tiempo de ejecución para clasificar si corresponde a una amenaza de seguridad (Inteligencia SIEM básica) y construir el objeto estructurado JSON final.
- **Acción concreta:** Escriba el bloque de procesamiento de archivos:

```
def process_lines_to_json(file_path, lines):
    payloads = []
    for line in lines:
        match = LOG_PATTERN.match(line)
        if match:
            data = match.groupdict()
            msg = data['message']

            # Inicializacion de variables estructuradas bajo logica SIEM
            event_type = "generic_system_event"
            username = "unknown"
            source_ip = None

            # Regla de correlacion para detectar intentos de fuerza bruta sobre SSH
            if "Invalid user" in msg or "Failed password" in msg:
                event_type = "authentication_failure"

            # Buscar patrones secundarios en el mensaje para extraer actor y origen
            user_match = re.search(r'for\s+(\w+)', msg)
            ip_match = re.search(r'from\s+([\d\.]+)', msg)

            if user_match: username = user_match.group(1)
            if ip_match: source_ip = ip_match.group(1)

            # Construccion del payload mapeado uno a uno con el Mapping de la Fase 3
            payload = {
                "@timestamp": datetime.utcnow().isoformat() + "Z", # Formato ISO estandar
```

```
        "log_file": file_path,
        "hostname": data['hostname'],
        "process": data['process'],
        "event_type": event_type,
        "username": username,
        "message": msg
    }
    if source_ip:
        payload["source_ip"] = source_ip

    payloads.append(payload)
return payloads
```

- **Explicación técnica inmediata:** Este bloque actúa como el analizador sintáctico (*Parser*). Convierte cadenas libres en variables discretas. Si el mensaje contiene fragmentos como "Invalid user", la regla de negocio eleva la clasificación a "authentication_failure". Luego, expresiones regulares secundarias (re.search) aíslan la dirección IP de origen de la conexión y la cuenta de usuario víctima. Todo se empaqueta en un formato clave-valor listo para serialización JSON.

5.5. Paso 4.5: Mecanismo de Transporte REST POST y Validación de Códigos HTTP

- **Objetivo del paso:** Despachar los documentos JSON estructurados hacia Elasticsearch por medio de la red de datos e inspeccionar la respuesta del servidor.
- **Acción concreta:** Incorpore la rutina de transmisión de red y validación de códigos de estado HTTP en el archivo:

```
def send_to_elasticsearch(payloads):
    for payload in payloads:
        try:
            # Enviar peticion estructurada POST conteniendo el payload JSON
            response = requests.post(ELASTIC_URL, headers=HEADERS, data=json.dumps(payload))

            # Validacion estricta del codigo de estado HTTP retornado por Elasticsearch
            if response.status_code == 201:
                print(f"[INGESTA EXITOSA] Evento de tipo '{payload['event_type']}' indexado")
            else:
                print(f"[ERROR DE MODELADO]Codigo: {response.status_code} - Detalles: {response.json()}")

        except requests.exceptions.ConnectionError:
            print("[FALLO CRITICO DE RED] No se pudo establecer canal con la API REST de Elasticsearch")
```

- **Explicación técnica inmediata:** El comando `requests.post` encapsula el JSON y lo transmite sobre un socket TCP dirigido al puerto 9200. Un código HTTP 201 `Created` garantiza de manera absoluta que el documento pasó las reglas de validación de mappings y ha sido guardado de forma permanente. Cualquier otro código (como 400 `Bad Request`) devela inconsistencias graves entre el tipo de dato enviado por Python y el Mapping de la Fase 3.

5.6. Paso 4.6: Integración del Bucle de Lectura y Ejecución Global

- **Objetivo del paso:** Unificar todos los módulos previos en una rutina principal que recorra de forma automatizada e inteligente el directorio de logs del sistema operativo Linux.
- **Acción concreta:** Cierre la edición de su script pegando el punto de entrada principal del código:

```
def main():
    print("Iniciando Agente Personalizado de Ingesta...")
    for root, dirs, files in os.walk(LOG_DIR):
        for file in files:
            file_path = os.path.join(root, file)

            # Aplicar filtros operativos de exclusion de rutas
            if is_binary(file_path) or (not file.endswith('.log') and file != 'auth.log'):
                continue

            try:
                with open(file_path, 'r', errors='ignore') as f:
                    lines = f.readlines()[-100:]
                    payloads = process_lines_to_json(file_path, lines)
                    send_to_elasticsearch(payloads)
            except Exception as e:
                print(f"No se pudo procesar el archivo {file_path}: {e}")

if __name__ == "__main__":
    main()
```

Guarde el archivo y ejecute el script desde la terminal de comandos de su VM:

```
python3 log_shipper.py
```

- **Resultado esperado de la Fase 4:** La terminal debe comenzar a arrojar mensajes continuos confirmando la lectura de bitácoras del sistema operativo: `[INGESTA EXITOSA] Evento de tipo 'generic_system_event' indexado correctamente`. Si se procesan logs de intentos fallidos de accesos, se visualizará el cambio dinámico de la etiqueta a `'authentication_failure'`.

- **Troubleshooting General de Ingesta:** Si la ejecución aborta inmediatamente con el error `ModuleNotFoundError: No module named 'requests'`, su entorno de Python carece de la librería cliente de red; instálela corriendo el comando del gestor de paquetes Linux: `sudo apt install python3-requests -y`.

Checkpoint de Validación 3

- ✓ El script de Python compila y lee el archivo de logs sin interrupciones.
- ✓ Códigos HTTP 201 `Created` observados de forma continua en la terminal.

6. Fase 5: Fundamentos de Query DSL y Validación Operativa vía REST API

Antes de abrir interfaces gráficas, un ingeniero de datos verifica la base de datos a nivel de protocolo HTTP puro. Aunque en Kibana usaremos una interfaz gráfica, debajo ocurre una traducción a **Query DSL** (el lenguaje de consultas nativo de Elasticsearch mediante JSON). Es vital entender cómo interactúan los tipos de datos:

- **Match (text):** Busca coincidencias parciales. Si busca `"match": {"message": "failed"}`, Elasticsearch analizará la frase completa y devolverá los logs que contengan esa palabra mediante el índice invertido.
- **Term (keyword):** Busca coincidencias exactas y binarias. `"term": {".event_type": ".authentication_fail"}` buscará únicamente el texto exacto sin pasar por tokenización.
- **Aggregations:** Permiten agrupar y computar métricas sobre los datos en disco. Por ejemplo, una agregación de tipo `terms` sobre el campo `source_ip` es lo que permite contar y crear el top de IPs atacantes.
- **Bool:** Permite combinar condiciones lógicas complejas (`must`, `must_not`, `should`).

6.1. Paso 5.1: Operaciones `_count` y `_search`

- **Objetivo del paso:** Utilizar consultas Query DSL crudas para confirmar la integridad y volumen de la base de datos distribuida de forma agnóstica.
- **Acción concreta:** Desde su computadora local o utilizando la herramienta **Dev Tools** de Kibana, ejecute las siguientes consultas HTTP REST secuenciales:

```
# 1. Contar cuantos documentos existen en el indice actualmente
curl -X GET "http://<IP_PUBLICA_VM>:9200/gcp-vm-logs/_count?pretty"
```

```
# 2. Buscar específicamente ataques usando una consulta de termino (term query) nativa
```

```
curl -X GET "http://<IP_PUBLICA_VM>:9200/gcp-vm-logs/_search?pretty" -H "Content-Type: ap
{
  "query": {
    "term": { "event_type": "authentication_failure" }
  }
},'
```

- **Explicación técnica:** `_search` interactúa con el motor Lucene interno para retornar los JSONs que nuestro script de Python ingestó, aplicando filtros directos sobre las estructuras binarias indexadas en la Fase 3.
- **Resultado esperado:** La API responderá con payloads JSON estructurados. En `_count` verá un número entero bajo el campo `count`". En `_search` verá los registros forenses crudos mapeados.

Checkpoint de Validación 4

- ✓ La API REST responde estructuralmente a operaciones de búsqueda con HTTP 200 OK.
- ✓ El conteo interno refleja numéricamente los registros procesados en Python.

7. Fase 6: Configuración Operativa y Análisis Forense en Kibana

7.1. Paso 6.1: Declaración Formal del Data View en el Panel de Administración

- **Objetivo del paso:** Crear una abstracción lógica de visualización (*Data View*) dentro de Kibana.
- **Acción concreta:**
 1. Abra su navegador web e introduzca la URL pública de visualización: `http://<IP_PUBLICA_TU_VM_GCP>`
 2. Navegue a **Management** → **Stack Management** → **Data Views**.
 3. Presione el botón azul: **Create data view**.
 4. Defina el nombre exacto: `gcp-vm-logs*` (El asterisco actúa como comodín).
 5. En **Timestamp field**, seleccione obligatoriamente: `@timestamp`.
 6. Haga clic en: **Save data view to Kibana**.
- **Resultado esperado:** La interfaz gráfica se actualizará mostrando una lista detallada con todos los campos tipificados que declaramos en la Fase 3. Esto comprueba la conexión lógica exitosa.

7.2. Paso 6.2: Inferencia Forense de Ataques en Tiempo Real (Discover)

- **Objetivo del paso:** Filtrar logs para aislar patrones de agresión maliciosa del flujo normal de administración del sistema operativo.
- **Acción concreta:** Navegue en el menú hacia **Analytics** → **Discover**. Asegúrese de seleccionar el Data View `gcp-vm-logs*`. Introduzca la siguiente expresión sintáctica en la barra de consultas de KQL y presione *Enter*:

```
event_type : "authentication_failure"
```

Ajuste el rango de tiempo de visualización en la esquina superior derecha seleccionando un intervalo de **Last 1 hour** o **Last 24 hours**.

- **Interpretación de ciberseguridad (Fuerza Bruta):** El estudiante observará un histograma de barras con picos de actividad verticales extremadamente densos y anormales (cientos de registros en pocos segundos). Al encontrarse expuesta con una IP pública en internet, los *bots* automatizados globales escanean la red perimetral de forma constante. Al detectar el puerto TCP 22 abierto, ejecutan ataques automatizados probando diccionarios enteros de credenciales. Cada barra masiva representa una ráfaga automatizada intentando adivinar contraseñas sobre usuarios comunes como `root`, `admin` o `test`.

7.3. Paso 6.3: Construcción de un Tablero de Respuestas Forenses (Dashboard)

- **Objetivo del paso:** Consolidar las métricas de seguridad descubiertas en una interfaz gráfica ejecutiva centralizada (*Dashboard*).
- **Acción concreta:**
 1. Navegue a la sección lateral del menú: **Analytics** → **Dashboard** → **Create dashboard**.
 2. Haga clic en: **Create visualization** (herramienta *Lens*).
 3. En el panel derecho de campos indexados, arrastre la variable `username` sobre el lienzo central.
 4. En el menú de tipo de gráfico izquierdo, cambie la vista predeterminada a un gráfico de pastel: **Pie**. Presione **Save and return**.
 5. Repita la operación arrastrando ahora el campo de red `source_ip` para generar una tabla que liste los principales actores agresores externos.
- **Explicación técnica inmediata:** La herramienta *Lens* ejecuta agregaciones del tipo `terms` en Elasticsearch de forma transparente. Cuenta la frecuencia de repetición exacta de términos idénticos en el campo mapeado como keyword y calcula proporciones estadísticas vectoriales para renderizar el gráfico. El pipeline está completo: transformamos texto plano oscuro en inteligencia visual.

Checkpoint de Validación Final

- ✓Data View configurado y acoplado con las series de tiempo de Elasticsearch.
- ✓Histograma de Kibana Discover refleja picos de actividad maliciosa en tiempo real.
- ✓Dashboard operativo consolidado mostrando vectores métricos del ataque.

8. Fase 7: Cuestionario de Análisis Forense Avanzado (Entregable Obligatorio)

Para validar la correcta comprensión del modelo de datos y la capacidad analítica de ingeniería, el estudiante deberá interactuar de forma reflexiva con la interfaz de Kibana (consultas KQL en *Discover*) y con la API REST de Elasticsearch para responder las siguientes interrogantes. Las respuestas deberán incluirse en el reporte final, documentando las capturas, consultas y justificaciones técnicas utilizadas para hallar el dato:

1. **Volumen del Incidente:** Utilizando la barra de búsqueda en *Discover*, determine: ¿Cuál es el número exacto de eventos clasificados bajo el `event_type : ".authentication_failure"` registrados en su servidor? Indique la consulta KQL exacta que utilizó para aislar este dato.
2. **Identificación del Agresor Principal:** Construya una visualización temporal (o utilice la tabla de campos a la izquierda en *Discover*) para identificar cuál es la dirección IP (`source_ip`) desde la cual se originó la mayor cantidad de intentos fallidos. Una vez identificada, consulte esa IP en una base de datos pública de reputación de Internet (como *VirusTotal*, *AbuseIPDB* o *Whois*) e indique de qué país, coordenadas geográficas aproximadas y proveedor de infraestructura de red (ASN) proviene.
3. **Análisis del Diccionario de Fuerza Bruta:** Liste el *Top 5* de los nombres de usuario (`username`) que los atacantes intentaron vulnerar con mayor frecuencia. Analizando la naturaleza de estos nombres de usuario, explique analíticamente: ¿El ataque fue dirigido específicamente a un usuario real de su sistema que el atacante conocía de antemano, o se trató de un escaneo automatizado global con un diccionario genérico por defecto? Justifique su respuesta basándose en los nombres observados.
4. **Aislamiento Temporal de Ráfagas:** Observe el histograma de eventos en *Discover*. Identifique el pico más alto de ataques y haga un "zoom" temporal controlado en la interfaz (arrastrando el cursor sobre esas barras densas). ¿En qué rango horario exacto (especificando hora, minuto y segundo de inicio y fin) se produjo la mayor ráfaga concentrada de intentos de acceso? ¿Cuántos intentos ocurrieron por segundo en el punto crítico?
5. **Auditoría de Ingesta vía REST API:** Abandone la interfaz gráfica por un momento. Abra su terminal Linux en la VM o la consola *Dev Tools* de Kibana y ejecute una consulta HTTP GET hacia el endpoint `_count` para determinar matemáticamente cuántos documentos en total posee el índice `gcp-vm-logs`. Copie y pegue el payload JSON de respuesta completo devuelto por el servidor e indique el valor de la clave `count`.

6. **Análisis de Línea de Base Operativa (Baseline):** Escriba una consulta KQL o realice un filtro en Kibana para aislar todos los eventos que NO correspondan a fallos de autenticación (es decir, los eventos clasificados inicialmente como `generic_system_event`). ¿Qué porcentaje del total de logs representan los eventos operativos normales frente a los eventos de ataque? Explique la importancia analítica de mantener una línea de base operativa en un SOC.
7. **Evasión Forense y Fragilidad del Parser (Análisis de Código):** Si un atacante malicioso sofisticado ejecutara un intento de inicio de sesión SSH utilizando de forma malintencionada un nombre de usuario que contenga espacios o caracteres no alfabéticos (por ejemplo, `."admin root" o inv@1$d`), ¿cómo afectaría esto al parsing de la expresión regular (`LOG_PATTERN`) compilada en la Fase 4 de su script de Python? Examine el código minuciosamente y deduzca qué valores tomarían las variables `username` y `event_type`. ¿Se indexaría el log correctamente o rompería el script?
8. **Mecanismo de Mitigación Perimetral Automático:** Basándose de forma estricta en la dirección IP del atacante principal descubierta en la Pregunta 2, investigue y escriba el comando de la herramienta de consola `gcloud compute firewall-rules` exacto que aplicaría en la infraestructura de Google Cloud para bloquear de manera inmediata, incondicional e imperativa (*DENY*) todo el tráfico de red entrante proveniente de ese actor hostil hacia cualquier puerto de su VM.

9. Problemas Reales Encontrados (Troubleshooting de Ingeniería)

Este laboratorio se diseñó para enfrentar al estudiante con la realidad operativa de la infraestructura distribuida. A continuación se documenta el comportamiento anómalo real diagnosticado durante las fases prácticas:

- **ERR_CONNECTION_TIMED_OUT (Bloqueo de Red):** Ocurrió constantemente al intentar conectar Kibana o la API REST desde la máquina local. No significaba que los contenedores estuvieran caídos, sino que el Firewall perimetral de GCP (Fase 1) descartaba silenciosamente los paquetes TCP entrantes al no coincidir exactamente la etiqueta semántica `elk-server` asignada a la instancia de VM.
- **Error de Localhost (Connection Refused en Ingesta):** El puerto físico estaba abierto en GCP, pero la interfaz gráfica externa rechazaba las conexiones. Causa: Docker levantó la aplicación escuchando por omisión de configuración únicamente en la interfaz de bucle cerrado local `127.0.0.1` (*loopback*) del anfitrión, en lugar de enrutarse de forma global a la dirección comodín `0.0.0.0`, aislando el servicio del tráfico perimetral.
- **El Mito del Estado "Yellow" en la Salud del Clúster:** Al consultar el estado del motor a través del endpoint administrativo `/_cluster/health`, la variable del estatus retornaba consistentemente un color amarillo. Muchos asumieron un fallo crítico en la base de datos Lucene. La realidad de arquitectura es que Elasticsearch asignaba correctamente el 100 % de los fragmentos primarios, pero

al operar en un esquema académico de nodo único (*Single-Node Clúster*), lanzaba una advertencia de redundancia debido a que físicamente no disponía de un segundo nodo esclavo donde replicar de forma segura los fragmentos de réplica.

- **Caída de Contenedores por Falta de RAM (OOM - Exit 137):** Máquinas virtuales parametrizadas con dimensiones excesivamente ajustadas en GCP (como tipos de instancia *e2-micro* que ofrecen únicamente 1GB de RAM) sufren abortos de procesos abruptos. La Máquina Virtual de Java (JVM) de Elasticsearch requiere asignaciones estables de memoria en el montón (*Heap Memory*). Al verse desbordado, el subsistema de gestión de memoria del Kernel de Linux activa el proceso *Out Of Memory Killer (OOM Killer)* y aniquila de forma forzada el contenedor Docker, arrojando el código de salida 137.
- **Colisión de Esquemas Dinámicos (*document_parsing_exception*):** Ocurrió al saltarse intencionalmente la Fase 3 de mappings explícitos. Si el script de Python enviaba una cadena de texto en un campo y posteriormente el flujo de logs Linux registraba una variable estructurada o una dirección IP mal formateada en ese mismo campo, el núcleo de Elasticsearch congelaba la transacción y lanzaba un rechazo de modelado, impidiendo la indexación duradera de las evidencias de seguridad para preservar la consistencia inmutable de sus índices invertidos.

10. Conclusión

A través de este laboratorio práctico, transitamos de manera completa por el ciclo de vida del dato: aprovisionamiento de políticas de red en la nube, despliegue de infraestructura efímera contenerizada, manipulación de la API REST de Elasticsearch, desarrollo de agentes livianos con Python utilizando lógica de parsing Regex, y visualización ejecutiva orientada a la detección de intrusos. Entender que el modelado analítico previo de datos es lo que verdaderamente habilita a un equipo de seguridad para cazar amenazas es la diferencia fundamental entre un administrador de sistemas tradicional y un ingeniero de datos en ciberseguridad.

11. Bibliografía

Referencias

- [1] Logz.io. *Python Logs to ELK / Elastic Stack*. URL: <https://logz.io/blog/python-logs-elk-elastic-stack/>
- [2] Medium (swlh). *Python Async Logging to an ELK Stack*. URL: <https://medium.com/swlh/python-async-logging-to-an-elk-stack-35498432cb0a>
- [3] Elastic NV. *Ingest data with Python on Elasticsearch Service*. URL: <https://www.elastic.co/docs/manage-data/ingest/ingesting-data-from-applications/ingest-data-with-python-on-elasticsearch-service>

- [4] Sematext. *ELK Stack Guide*. URL: <https://sematext.com/guides/elk-stack/>
- [5] Khalifa Farhat (Medium). *Building a Secure ELK Stack with Log Analysis and Brute Force Detection*. URL: https://medium.com/@khalifa_farhat/building-a-secure-elk-stack-with-log-analysis-and-brute-force-detection-24656db791d7